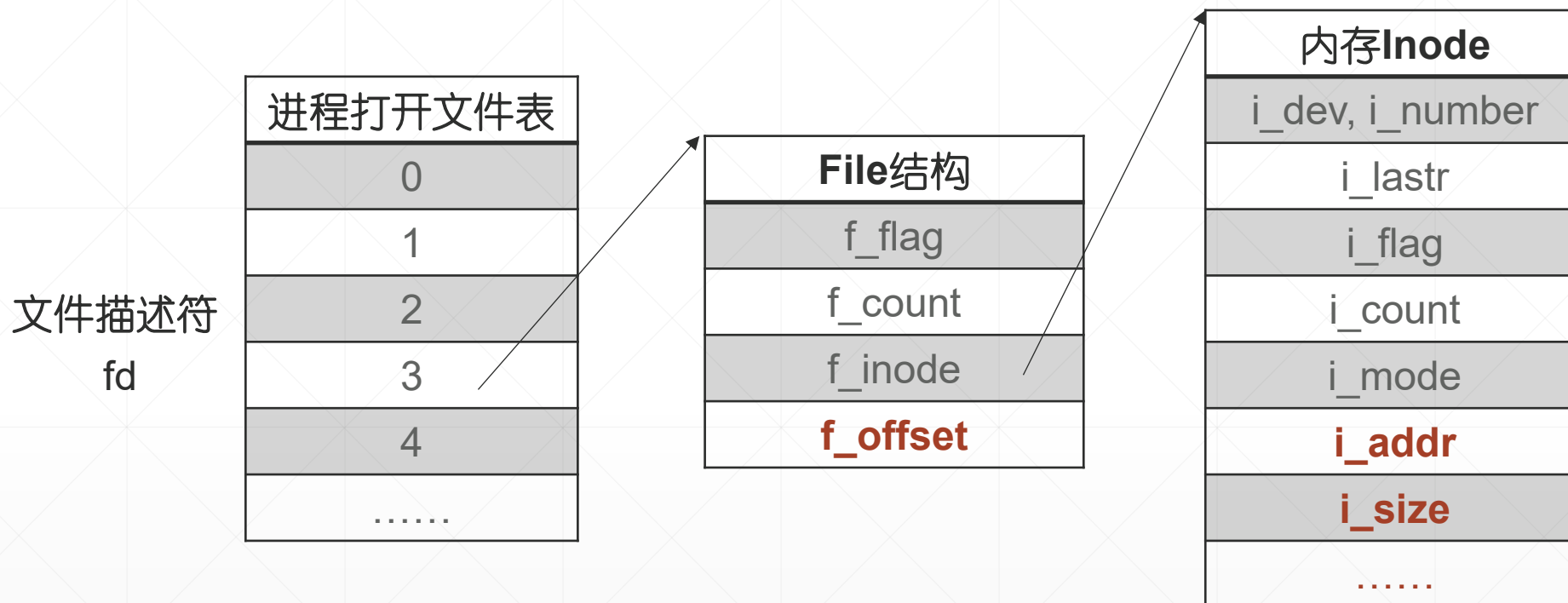


三、进程访问数据文件，需要使用打开文件结构



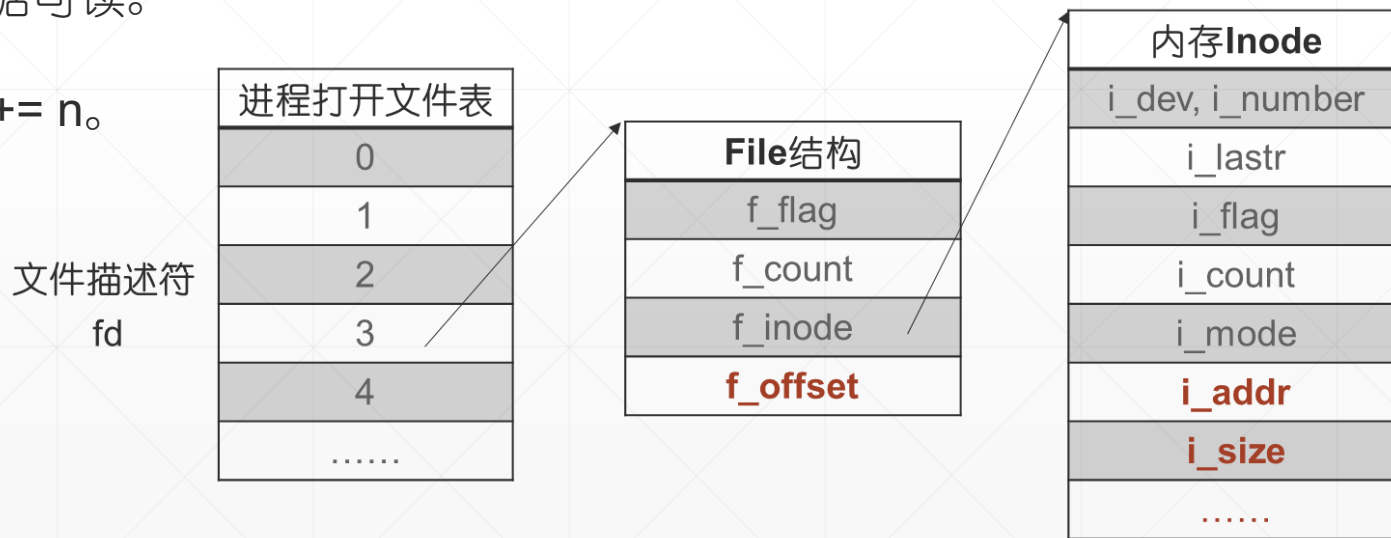
```
n=read(fd,buf,nbytes);  
n=write(fd,buf,nbytes);  
seek(fd,offset,ptrname);
```

1、 $n = \text{read}(fd, buf, nbytes);$

读 fd 引用的文件，从 f_offset 开始，读 $nbytes$ 字节，存入用户空间，首地址为 buf 的内存单元。
返回值 n 是 实际读入的字符数

- (1) $n < nbytes$ ，读至文件尾部。
- (2) $n = 0$ ，EOF (End Of File)，没有数据可读。

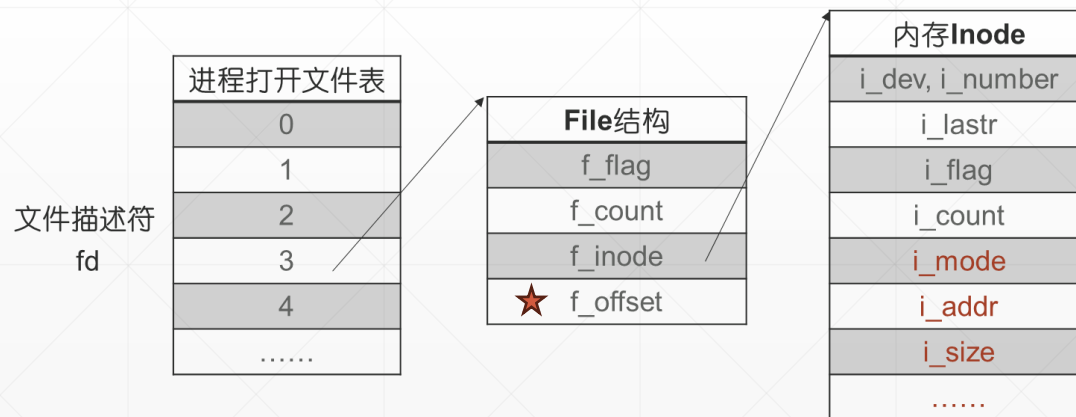
读操作结束后调整文件读写指针， $f_offset += n$ 。



2、 $n = \text{write}(fd, buf, nbytes);$

将用户空间首地址为buf的一块数据写入 fd 引用的文件。从f_offset开始，向后写nbytes字节。
返回值 n 是 实际读入的字符数，不出意外，一定是nbytes。

写操作结束后调整文件读写指针， $f_offset += n$ 。
写操作执行过程中，文件系统为它分配数据块、索引块。
若文件长度增长，更新 i_size。

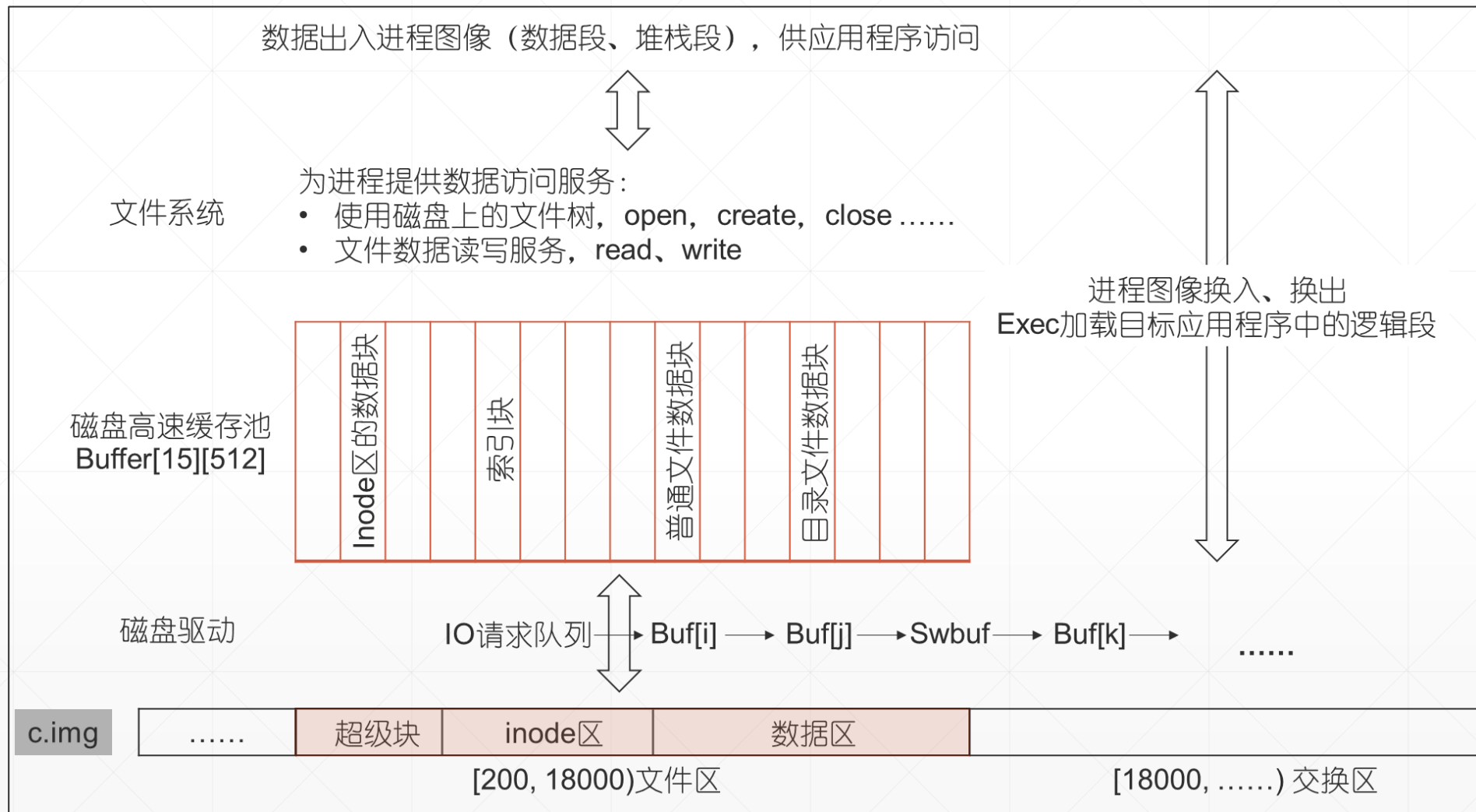


Unix系统架构 (二)

文件系统为进程提供数据访问服务时，需要使用磁盘上的元数据和文件数据。

文件系统访问磁盘高速缓存读写磁盘数据。

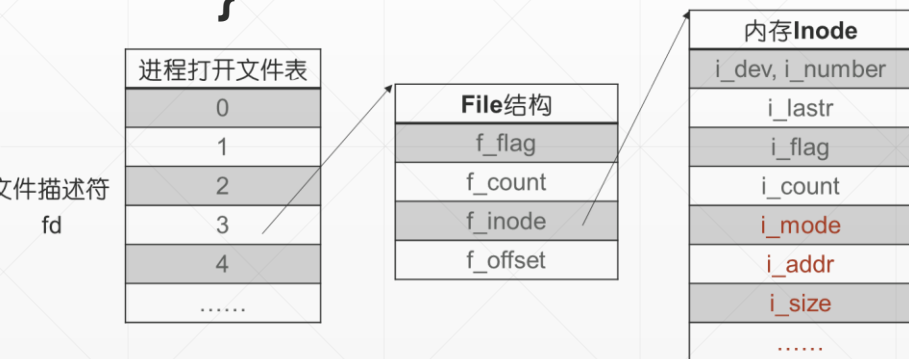
- 缓存不命中时，执行IO操作从磁盘调数据。
- 更新后的超级块、Inode同步写回磁盘。
- 更新后的数据盘块延迟、异步写回磁盘。



3、read/write 系统调用的实现

```
void FileManager::Read()
{
    this->Rdwr(File::FREAD);
}
```

```
void FileManager::Write()
{
    this->Rdwr(File::FWRITE);
}
```



```
void FileManager::Rdwr( enum File::FileFlags mode )
{
```

```
    File* pFile;
    User& u = Kernel::Instance().GetUser();

    1、检查文件的操作方式
    pFile = u.u_ofiles.GetF(u.u_arg[0]); // fd引用的File结构
    if ( (pFile->f_flag & mode) == 0 ) // FREAD & FWRITE为0, 不可以写读打开的文件
    {
        u.u_error = User::EACCES;
        return;
    }

    2、设置IO参数
    u.u_IOParm.m_Base = (unsigned char *)u.u_arg[1]; /* 用户空间首地址 */
    u.u_IOParm.m_Count = u.u_arg[2]; /* 要求读/写的字节数 */
    u.u_IOParm.m_Offset = pFile->f_offset; /* 数据在文件中的偏移量 */
    u.u_segflg = 0; /* User Space I/O, 读入的内容要送数据段或用户栈段 */

    3、读写文件
    pFile->f_inode->NFlock(); // 锁住内存Inode
    if ( File::FREAD == mode )
        pFile->f_inode->Readl(); // 读文件
    else
        pFile->f_inode->Writel(); // 写文件
    pFile->f_inode->NFrele(); // 内存Inode解锁

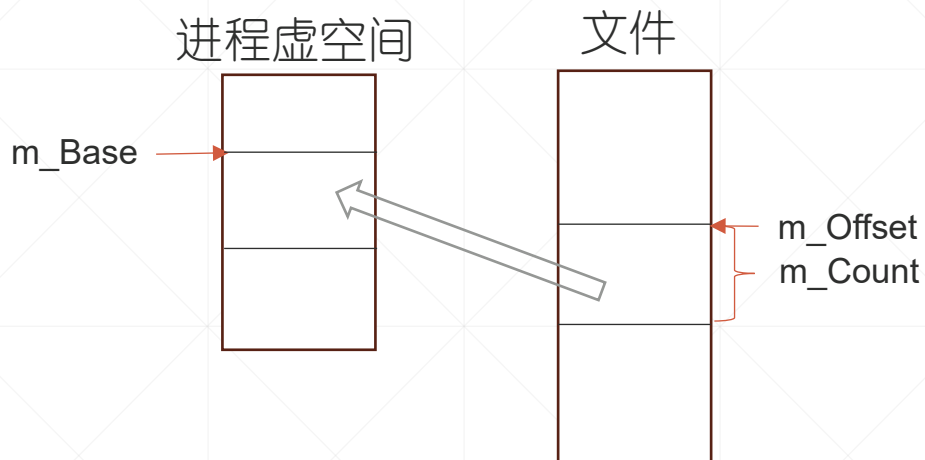
    4、调整文件的读写指针
    pFile->f_offset += (u.u_arg[2] - u.u_IOParm.m_Count);

    5、返回实际读写的字符数
    u.u_ar0[User::EAX] = u.u_arg[2] - u.u_IOParm.m_Count;
}
```

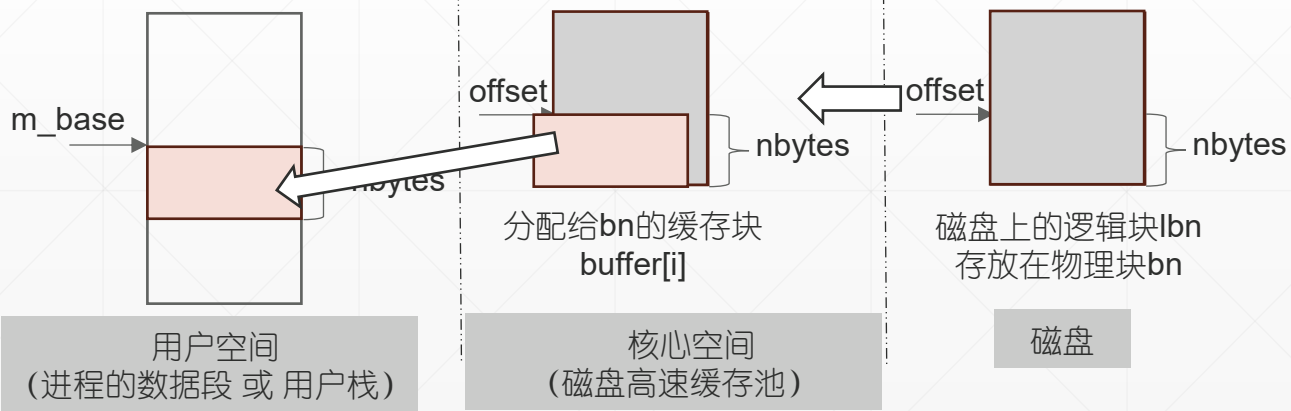
01	1, 读操作合法
10	2, 写操作合法
11	1和2都合法

逐个字符块读写文件
每读写一块, 调整一次IO参数

4、Readl()

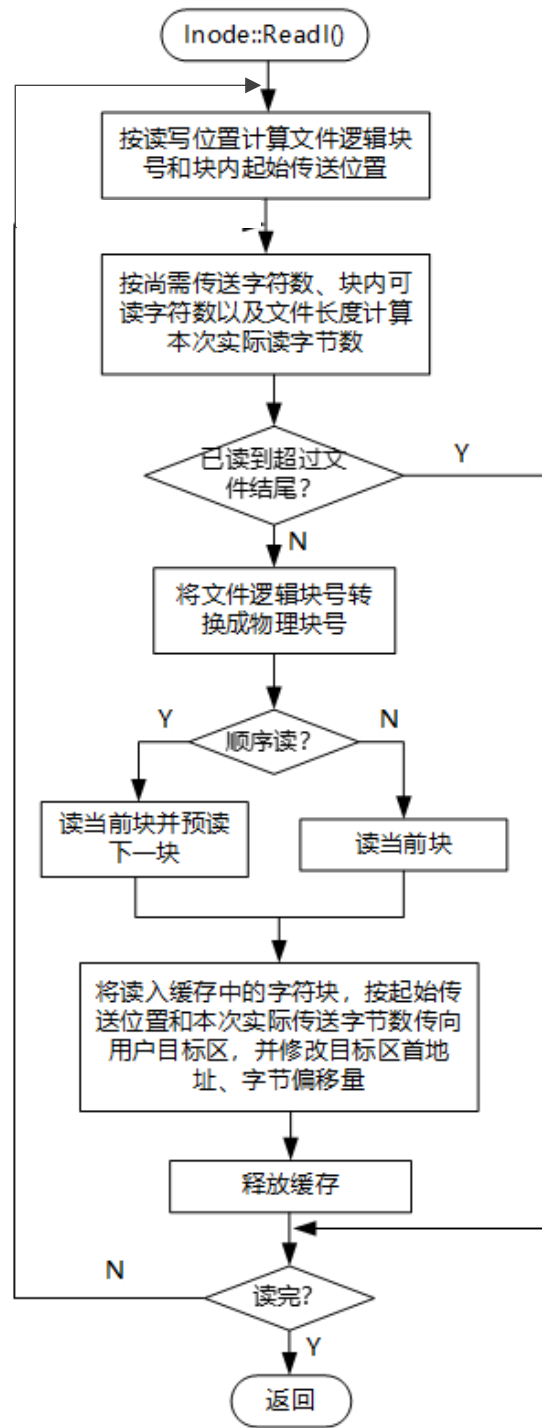


每次读一块（逻辑块号lbn），把这块中需要的数据全读出来（从块内偏移量offset开始，读nbytes字节），送用户空间（m_base）

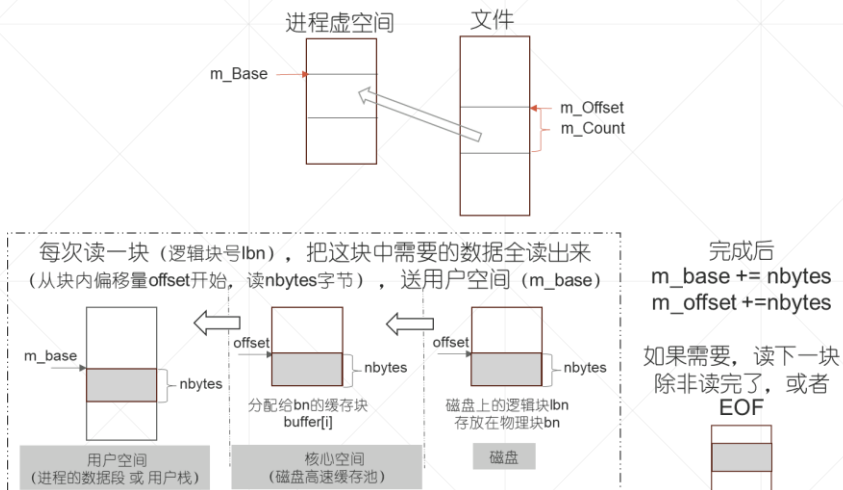


完成后
 $m_base += nbytes$
 $m_offset += nbytes$

如果需要，读下一块
 除非读完了，或者
 EOF



代码注释

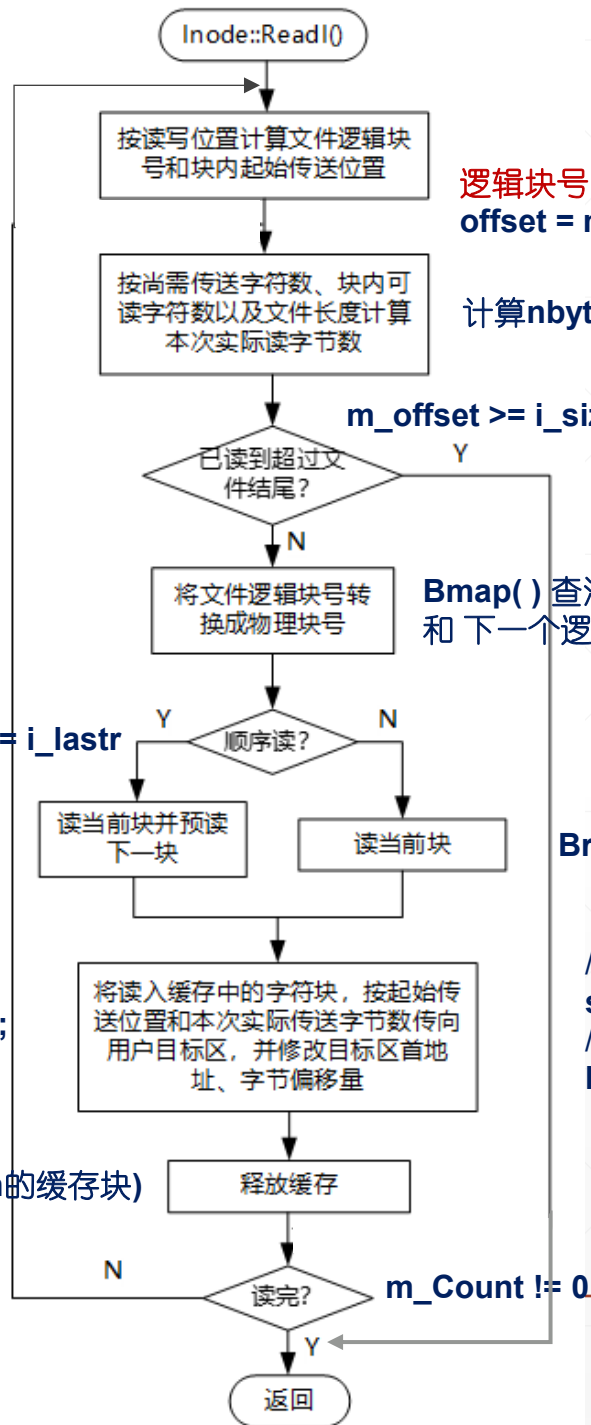


Breada(bn, rblkno) (有IO)

u.u_IOParm.m_Base += nbytes;
u.u_IOParm.m_Offset += nbytes;
u.u_IOParm.m_Count -= nbytes;

Brelse(bn的缓存块)

lbn == i_lastr



逻辑块号 lbn = m_offset / 512

offset = m_offset % 512 // 这个逻辑块中, 读操作的起始位置

计算nbytes, 这个数据块中需要读入的字节数

m_offset >= i_size (EOF)

Bmap() 查混合索引表计算 lbn 的物理块号 bn
和下一个逻辑块的物理块号 rblkno。

Bread(bn) (有IO)

// 计算数据在缓存中的首地址
start = pBuf->b_addr + offset;
// 把缓存块中的nbytes字节送至用户区
IOMove(start, m_Base, nbytes);
i_lastr = lbn

m_Count != 0

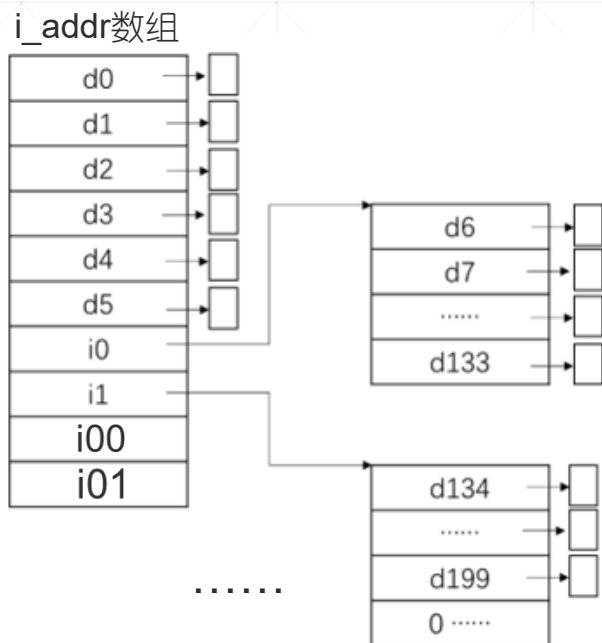
读操作的开销（一个块，不考虑预读）

- 地址映射，逻辑块号lbn→物理块号bn
- 数据复制，磁盘→缓存池→用户空间

表：地址映射开销

偏移量	逻辑块号	映射关系存放位置	地址映射IO开销
[0, 6*512)	0 ~ 5	内存, Inode	0
[6*512, 134*512)	6 ~ 133	一次索引块 i0	Bread(i0), 1次
.....	133 ~ 261	一次索引块 i1	Bread(i1), 1次
如果用到2次索引块, IO 2次, 先读2次索引块i00、再读1次索引块			

注1：文件打开后，inode 在主存里，直至 close。
 注2：索引块读入磁盘高速缓存后，可复用。命中



- 习题：成功打开文件fileA后，系统依次访问偏移量为 1, 100, 1000, 4096, 5120, 5200 的字节。IO多少次？已知，磁盘高速缓存中没有文件A的索引块和数据块。

偏移量	逻辑块号	是否需要读入索引块	是否需要读入逻辑块	IO次数
1	0	n	y	1
100	0	n	n	0
1000	1	n	y	1
4096	8	y	y	2
5120	10	n	y	1
5200	10	n	n	0

消耗5个缓存块

预读

- 如果目标文件顺序读，即 $lbn = i_lastr + 1$ ，同步读入当前块 lbn 时，将下个逻辑块 $lbn + 1$ 异步读入缓存池
 - 检索混合索引树，计算 lbn 和 $lbn + 1$ 的物理块号： $bn, rablkn$ 。
 - 将 bn 读入缓存池（确保缓存池中有 bn 数据块）：
 - 缓存命中？ go
 - 不命中：
 - $GetBlk(rablkn)$ 分配缓存块 $Buffer[i]$
 - $Buf[i]$ 置读标识，送 IO 请求队列
 - 将 $rablkno$ 读入缓存池（异步）：
 - $GetBlk(rablkn)$ 分配缓存块 $Buffer[j]$
 - 缓存命中？
 - Y: $Brelse(&Buf[j])$ ，解锁
 - N: $Buf[j]$ 置异步标识、读标识，送 IO 请求队列（IO 完成后，磁盘中断处理程序解锁预读块）
- $Bread(bn)$ 同步等待 IO 操作结束 & 锁住 $Buffer[i]$

预读 $Breada(bn, rablkn)$

预读的好处

- T时刻，进程PA执行如下操作顺序访问磁盘文件A。
文件A总长2个数据块，0#、1#数据块分别存放在50#磁道，扇区号：n、n+2。
执行read系统调用读文件A 0~511字节，访问n#扇区
.....处理读入的新数据（文件A的0~511字节）.....
执行read系统调用读文件A 512~1024字节，访问n+2#扇区
.....处理读入的新数据（文件A的512~1024字节）.....

并发的进程B执行如下操作访问存放在10000#磁道的另一个文件B，读写其中的某个数据块（扇区号m）。
执行read系统调用读文件B，访问 m#扇区

- 磁盘IO使用FCFS调度算法。假设，自由缓存队列有很多不脏的缓存块，n#、n+2#，m#扇区缓存不命中。磁头当前位于0#磁道。PA先执行。问（1）采用预读，服务这3个磁盘扇区，磁臂移动总距离



PA执行第1个read系统调用：将同步读请求n 和 异步读请求n+2送入IO请求队列后，入睡等待n#扇区读操作完成。
PA放弃CPU后，PB执行，将IO请求m送入IO请求队列。

上图是这些操作完成后IO请求队列的样子。完成这3个请求，磁臂移动 $10000-50=9950$ 根磁道。

此外，我们看PA执行第2次read系统调用的细节：n#扇区IO完成后，第1个read系统调用返回。处理完新数据后，PA开始执行第2个read系统调用，它入睡等待n+2#扇区IO完成。。。IO完成后，磁盘中断处理程序解锁 n+2# 数据块，唤醒PA使用预读块数据。

预读块IO几乎没有定位开销（寻道延迟和旋转延迟）。所以，第2次read系统调用，PA睡眠时间很短。

- T时刻, 进程PA执行如下操作顺序访问磁盘文件A。
文件A总长2个数据块, 0#、1#数据块分别存放在50#磁道, 扇区号: n、n+2。

执行read系统调用读文件A 0~511字节, 访问n#扇区
.....处理读入的新数据 (文件A 的0~511字节)
执行read系统调用读文件A 512~1024字节, 访问n+2#扇区
.....处理读入的新数据 (文件A 的512~1024字节)

并发的进程B执行如下操作访问存放在10000#磁道的另一个文件B, 读写其中的某个数据块 (扇区号m) 。

执行read系统调用读文件B, 访问 m#扇区

- 磁盘IO使用FCFS调度算法。假设, 自由缓存队列有很多不脏的缓存块, n#、n+2#, m#扇区缓存不命中。磁头当前位于0#磁道。PA先执行。问 (2) 不预读, 服务这3个磁盘扇区, 磁臂移动总距离



PA执行第1个read系统调用: 将同步读请求n 送IO请求队列, 入睡等待n#扇区读操作完成。

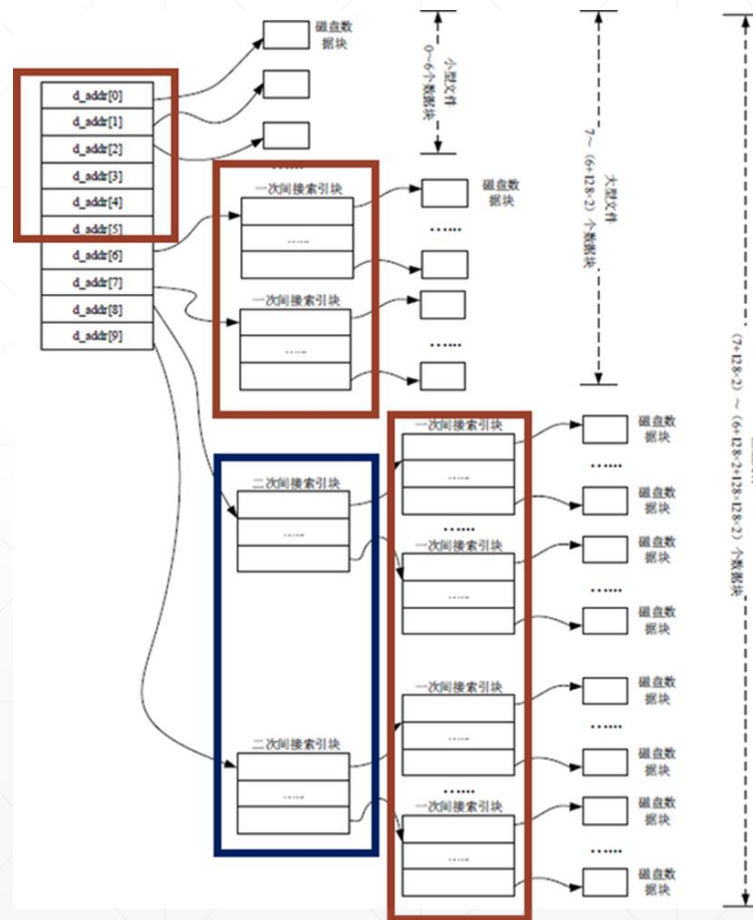
PA放弃CPU后, PB执行, 将IO请求m送入IO请求队列。未来, PA执行第2个read系统调用: 会将同步读请求n+2 送IO请求队列。上图是这些操作完成后IO请求队列的样子。完成这3个请求, 磁臂移动 $2 * (10000 - 50)$ 根磁道。

~~相邻扇区IO请求没法挨在一起, 整个磁臂移动轨迹有很多折返, 磁盘吞吐率下降了。~~

总结, 预读的好处: 在进程执行顺序读操作时, 预读可以减小磁臂移动总长度, 缩短读操作耗时。
前提: 文件系统会尽量为相邻逻辑块分配相邻的物理块。

顺序读操作，何时放弃预读？

- 同步块bn缓存命中时，构造预读块，价值大嘛？
- 如果计算预读块的物理块号rablkno需要读入索引块，可以考虑放弃预读。



提问：考虑预读，顺序读文件A，读lbn==5的逻辑块时，有必要预读吗？

5、Writel()

逻辑块号 $lbn = m_offset / 512$
 $offset = m_offset \% 512$

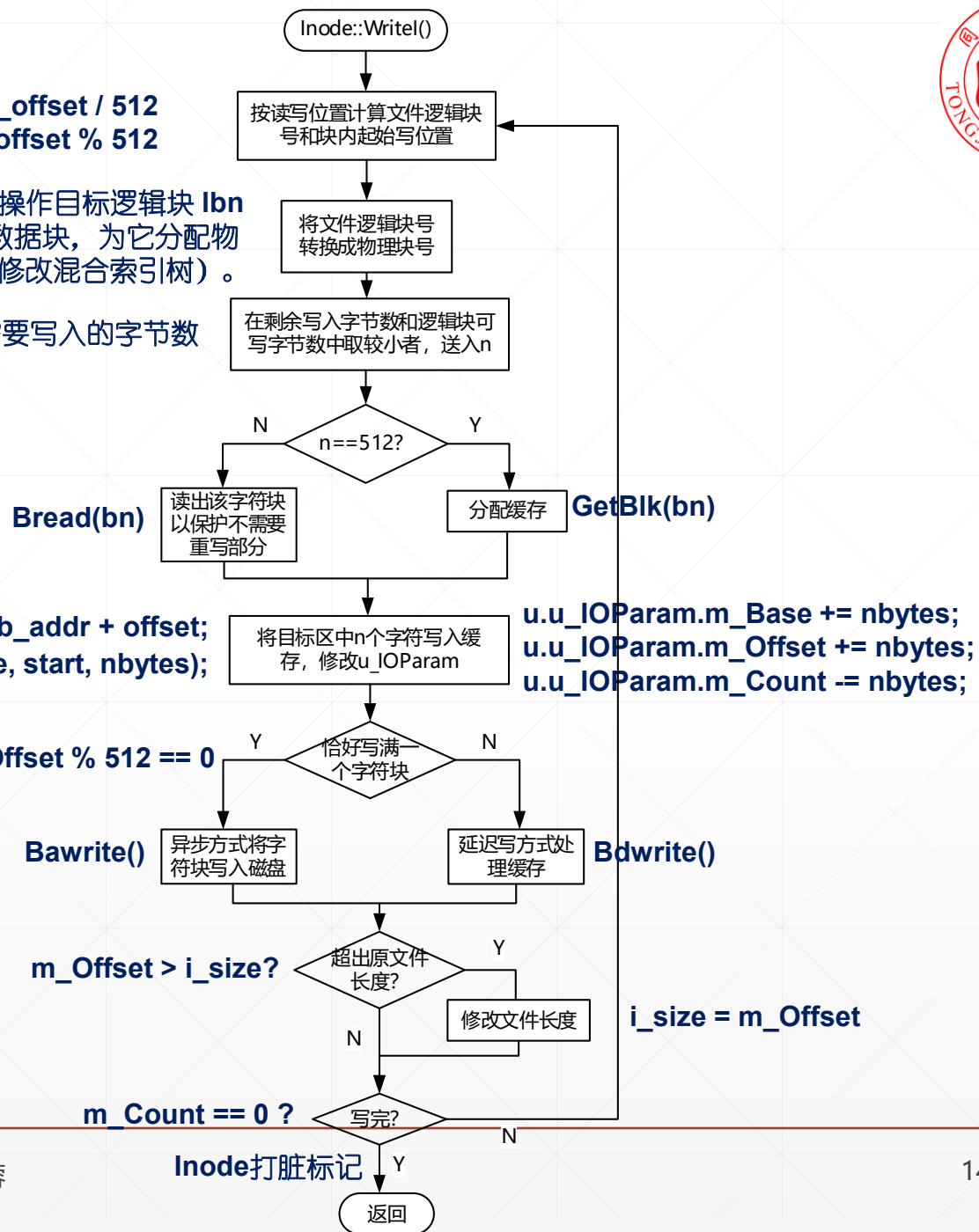
Bmap() 查混合索引表计算写操作目标逻辑块 lbn 的物理块号 bn 。若 lbn 是新数据块，为它分配物理块，必要时，分配索引块（修改混合索引树）。

计算 n ，数据块中需要写入的字节数

`unsigned char* start = pBuf->b_addr + offset;`
`IOMove(m_Base, start, nbytes);`

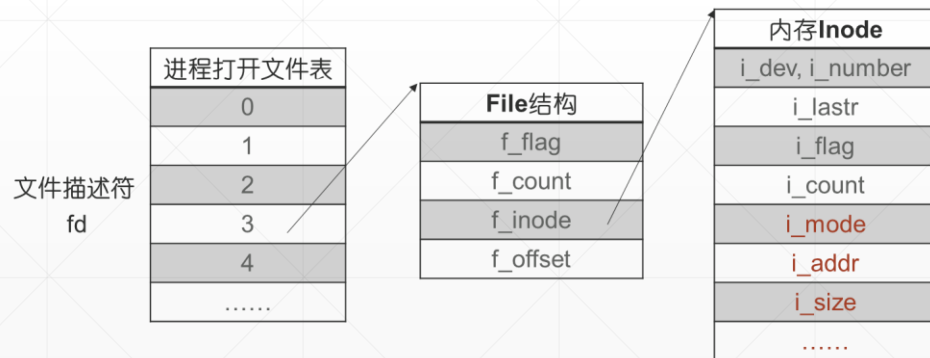
`u.u_IOParam.m_Base += nbytes;`
`u.u_IOParam.m_Offset += nbytes;`
`u.u_IOParam.m_Count -= nbytes;`

$m_Offset \% 512 == 0$



6、seek()

`seek(fd,offset,ptrname);` // 调整文件读写指针



```
void FileManager::Seek()
{
    .....
    int fd = u.u_arg[0];    // fd

    pFile = u.u_ofiles.GetF(fd); // File结构

    .....
    int offset = u.u_arg[1];    // offset

    .....
    switch ( u.u_arg[2] ) // ptrname
    {
        /* 读写位置设置为offset */
        case 0:
            pFile->f_offset = offset;
            break;

        .....

    }
}
```

习题 1

一、完整的文件读写过程

1、

```
int fd = open("/usr/ast/Jerry", 3); //以可读可写方式打开文件
char data[300];
seek(fd, 500, 0); //将文件读写指针定位到第500字节
int count = read (fd, data, 300); //从文件读300字节到data
count = write(fd, data, 300); //从 data 写 300 字节到文件
```

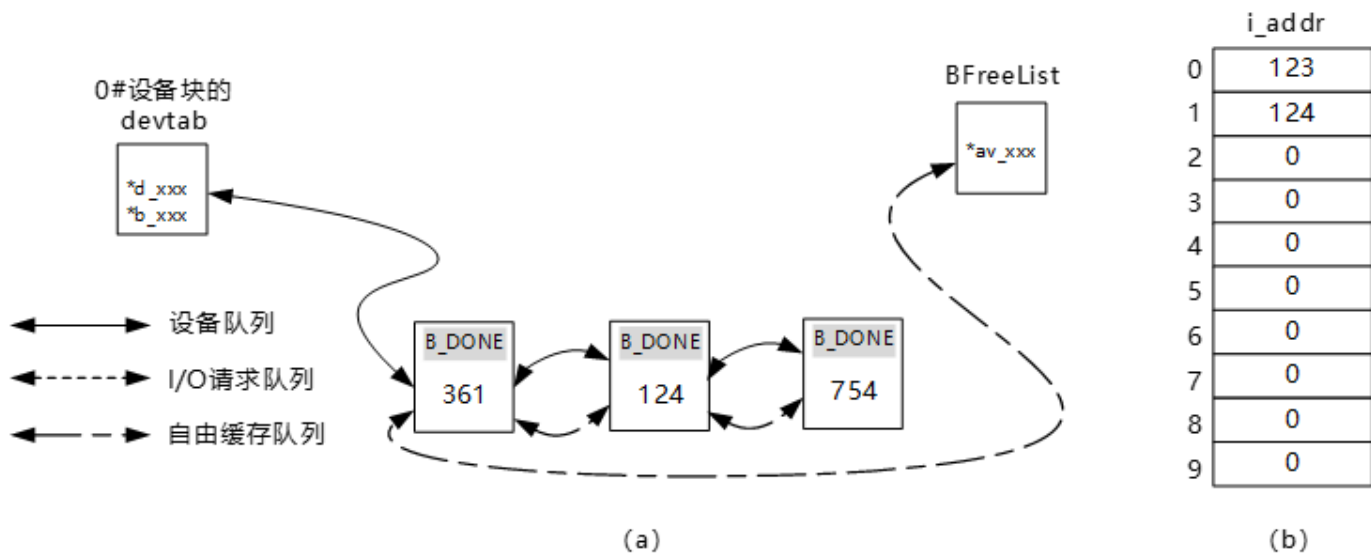


图 1、缓存队列 和 Jerry 文件的索引表

习题 2:

3、open 系统调用结束后，读下图中的文件 file1，假设所有数据块和缓存块缓存不命中，不考虑预读。进程依次执行以下读操作，回答问题（1）读 3#逻辑块，几次 IO？（2）读 6#逻辑块，几次 IO？（3）读入 6#逻辑块之后，接着读 100#逻辑块，几次 IO？（4）随后，写 198#逻辑块中的第 200#字节，几次 IO？

[参考答案]

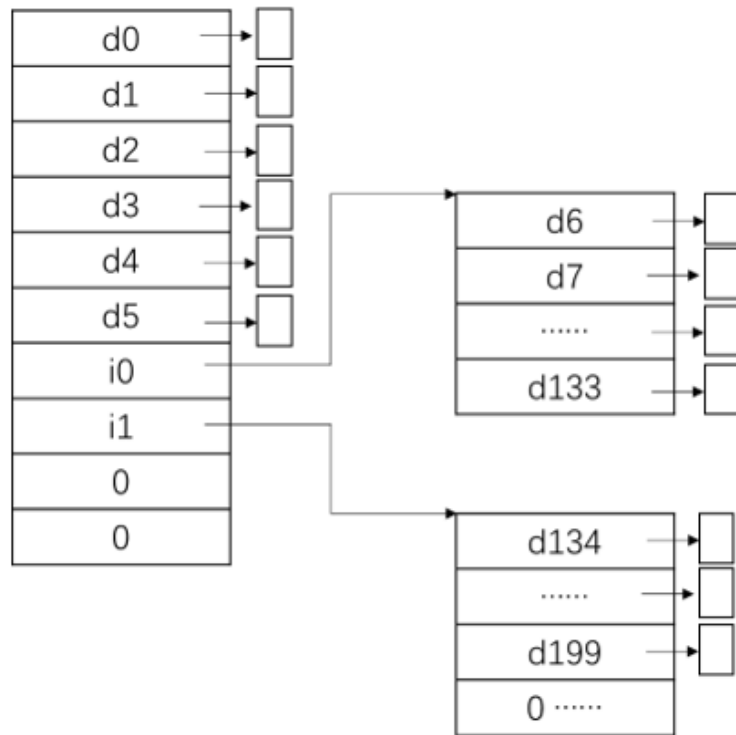
open 系统调用结束后，file1 的 Inode 在内存里。

（1）读 3#逻辑块，1 次 IO。0#~5#逻辑块的物理块号在 Inode 中，文件打开后在内存。所以，读 3#逻辑块 IO 一次，读入物理块 d3。

（2）读 6#逻辑块，2 次 IO。第一次读入物理块 i0、得到文件的第一个索引块，get 物理块号 d6。第二次读入物理块 d6，get 文件数据。

（3）读 100#逻辑块，1 次 IO。100#逻辑块的物理块号登记在第一个索引块，已经在内存里了。所以，只需一次 IO 读入物理块 d100。

（4）写 198#逻辑块中的第 200#字节，2 次 IO。一次读入物理块 i1，获得第 2 个索引块，查询得知 198#逻辑块存放在 d198#物理块中。第二次 IO，执行先读操作，将物理块 198 读入磁盘高速缓存。没有写 IO，因为这个逻辑块没有写满，就让它呆在缓存池里。



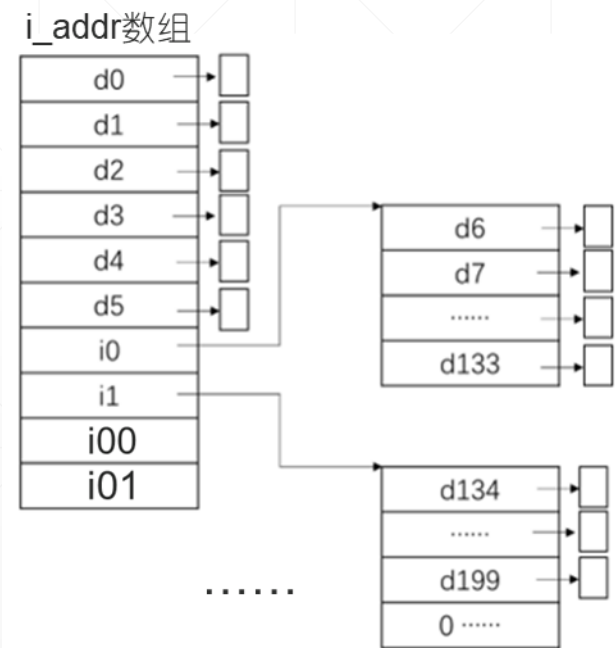
习题 3

- Unix V6 使用混合索引树（极不平衡的树结构）登记分配给文件的所有物理块号。为什么不用平衡树？

表：地址映射开销

偏移量	逻辑块号	映射关系存放位置	地址映射IO开销
[0, 6*512)	0 ~ 5	内存, Inode	0
[6*512, 134*512)	6 ~133	一次索引块 i0	Bread(i0), 1次
.....	133 ~ 261	一次索引块 i1	Bread(i1), 1次
如果用到2次索引块, IO 2次, 先读2次索引块i00、再读1次索引块			

注1：文件打开后, inode 在主存里, 直至 close。
 注2：索引块读入磁盘高速缓存后, 可复用。命中





四、open系统调用

`fd=open(name,mode);`

- `Namei()`: 线性目录搜索 (路径名`name`) , 确定文件的DiskInode号`<dev,number>`
- `IGet()`: 为`<dev,number>`分配内存Inode, 上锁
 - 不命中, 把DiskInode读入内存inode
- `Access()`: 权限检测
- 建立打开文件结构。分配File结构、文件描述符`fd`
- `IPut()`; 解锁内存Inode
- 返回 `fd`

1.1 线性目录搜索 Name1(), 查找已有文件

(1) 确定目录搜索起点

- 路径名以 / 开头, 绝对路径, 起点 **rootDirInode**
- 否则, 相对路径, 起点 **u_cdir**
- **work**=搜索起点, **work**是需要遍历的目录文件

(2) 用分隔符 / 分割路径名, 得到**路径名分量列表**

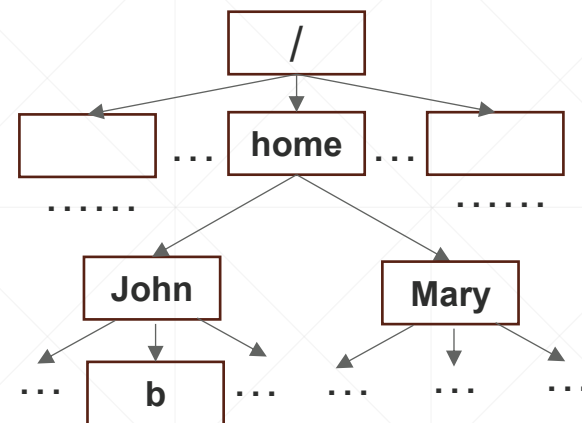
- "home\0" , "John\0" , "b\0"

(3) while (路径名分量列表非空)

```

{
    name=路径名分量列表第一个元素
    遍历work, 找 文件名 = 字符串name 的目录项dir
        . 找到, 读入ID==dir.inode的DiskInode, work=它的内存Inode
        . 没找到, 目录搜索失败, return null // 失败
    路径名分量列表第1个元素出列
}
return work // 成功, 返回目标文件的FCB (inode)

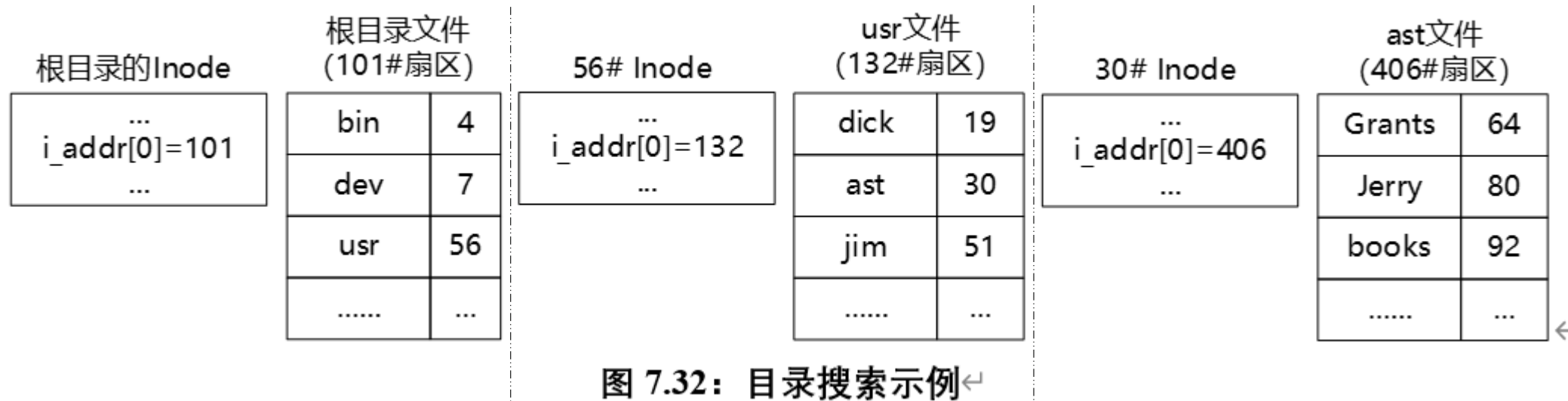
```



查找已有文件 /home/John/b

根目录文件 (1#文件) 搜索 “home”, 得inode号 5
 home目录文件 (5#文件) 搜索 “John”, 得inode号 26
 John家目录文件 (26#文件) 搜索 “b”, 得inode号 50
 返回 50 // 目录搜索成功, 返回b文件的inode号

例：线性目录搜索文件“/usr/ast/Jerry”。假设各级目录文件只有一个数据块，内容如图7.32所示。简述目录搜索过程，计算目录搜索需要执行的IO次数。





根目录的Inode

...
i_addr[0]=101
...

根目录文件
(101#扇区)

bin	4
dev	7
usr	56
.....	...

56# Inode

...
i_addr[0]=132
...

usr文件
(132#扇区)

dick	19
ast	30
jim	51
.....	...

30# Inode

...
i_addr[0]=406
...

ast文件
(406#扇区)

Grants	64
Jerry	80
books	92
.....	...



这是绝对路径名。搜索起点是根目录，1#文件。 “/usr/ast/Jerry”

目录搜索需要遍历目录文件，直至找到相关目录项。遍历过程分2步：（1）读入目录文件的 inode （2）逐个读入目录文件的数据块，遍历之。

本例，有3个路径名分量，目录搜索要遍历3个目录文件

（1）遍历根目录（1#文件），搜索文件名“usr”。成功，usr文件的 inode 号是56。

细节：根目录文件 Inode 常驻主存，无需读入，没有 IO。

遍历，需要读入 101#数据块，1次 IO

（2）遍历 usr 目录（56#文件），搜索文件名“ast”。成功，ast 文件的 inode 号是30。

细节：56# Inode 不在主存，需要读入，1次 IO。

遍历，需要读入 132#数据块，1次 IO

（3）遍历 ast 目录（30#文件），搜索文件名“Jerry”。成功，Jerry 文件的 inode 号是80。

细节：30# Inode 不在主存，需要读入，1次 IO。

遍历，需要读入 406#数据块，1次 IO

目录搜索成功，返回 Jerry 文件的 inode 号80。此次目录搜索，IO 5次，将5个磁盘数据块读入磁盘高速缓存。

评论：线性目录搜索开销巨大。（1）好慢（2）淘汰掉磁盘高速缓存中的5个 LRU 数据块。一定要想办法优化。

优化的方法：（1）尽量用相对路径名引用文件，路径短，会快好多。看下面的例子。

（2）文件系统设置目录项缓存，保留最近用过的目录项。

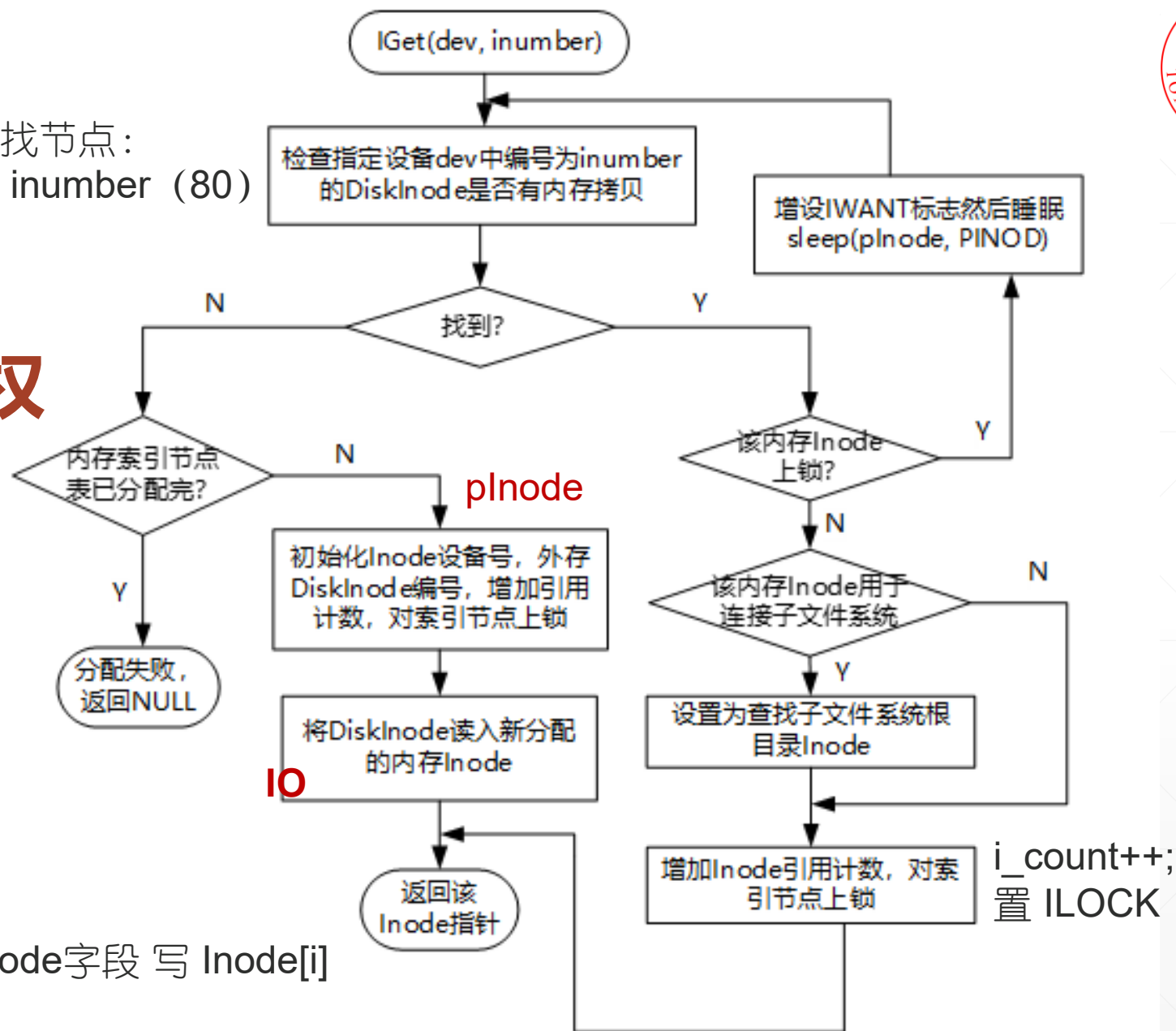
（3）目录文件中，目录项排序。

遍历内存Inode表，查找节点：
 $i_dev == dev; i_number == inumber$ (80)

1.2 IGet()获得80#DiskInode的使用权

```
plnode->i_dev = dev;
plnode->i_number = inumber (80) ;
plnode->i_flag = Inode::ILOCK;
plnode->i_count++;
plnode->i_lastr = -1;
```

```
buf = Bread( inumber/8 + 2 )
从 buf.b_addr+inumber%8*64 读取DiskInode字段 写 Inode[i]
```

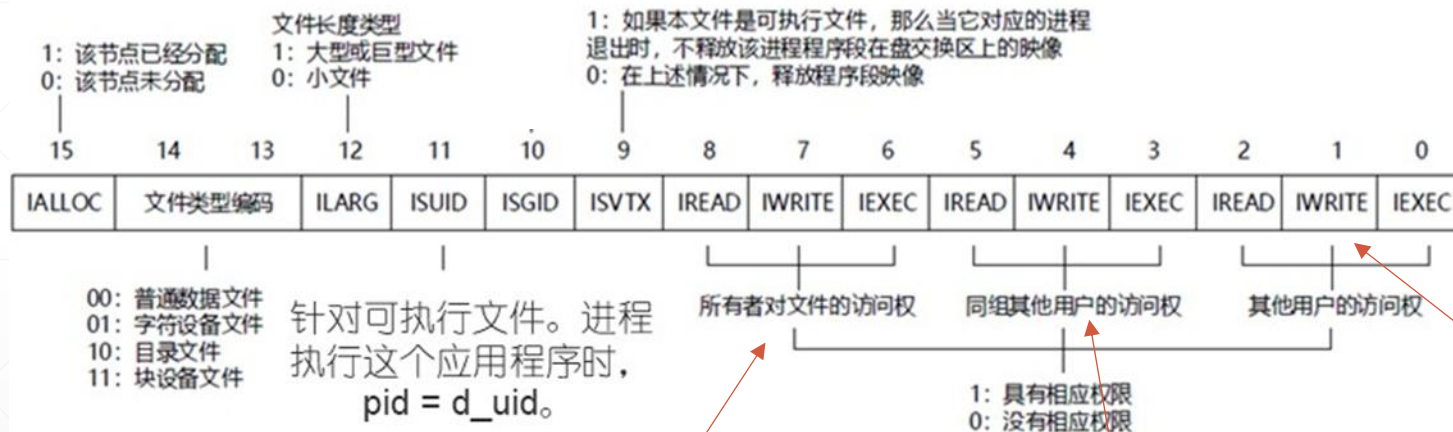


1.3 Access() 检查进程对文件是否有 mode请求的访问权限

fd=open(name,mode);

mode = FREAD, FWRITE, FREAD | FWRITE

读取 80#DiskNode 的
i_flag



`p_uid == i_uid`

读取现运行进程Process结构
`p_uid` 和 `p_gid`

`p_gid == i_gid`

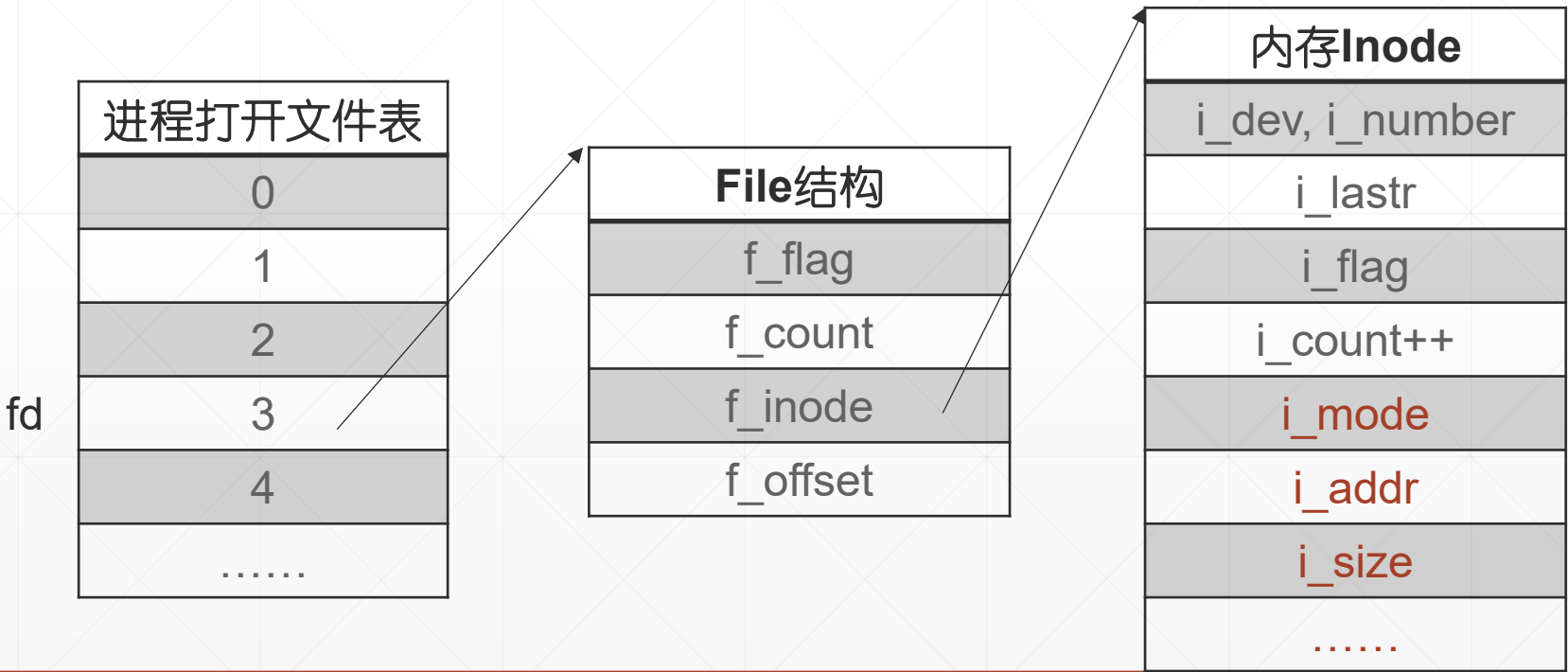


1.4 建立、初始化打开文件结构。分配File结构、文件描述符fd

```
fd=open(name,mode);
```

- 1、分配File结构（空闲标识：f_count == 0）
 - 2、从下标 0 开始，在进程打开文件表中寻找空闲表项（第1个为Null的表项）。fd是下标。
 - 3、初始化File结构
 - f_flag = mode
 - f_count++
 - f_offset = 0
 - 4、建立链接关系
- 返回 fd

i_lastr = -1





五、系统调用 `Creat(“file”, mode)`

- 线性目录搜索，打开新建文件的父目录文件 `Name1()`，分支2
- 父目录文件，进程有写权限嘛？
 - 没有，出错返回
- 父目录文件中有同名文件“file”？
 - 有，清空已有文件：打开目标文件Inode，从0#逻辑块开始，释放所有数据块。`i_addr`数组清0，`i_size=0`。
 - 没有，
 - 父目录文件中为新文件分配一个目录项
 - 分配一个DiskInode，`d_link=1`，`d_mode=mode`(Creat系统调用的第2个参数)，`d_uid=p_uid`，`d_size=0`；
 - 将DiskInode号和文件名写入新目录项
- `IPut()`父目录文件的内存Inode
- 为新建文件建立打开文件结构。File结构的 `f_mode=FWRITE`，`f_offset=0`
- 返回 `fd`



六、系统调用 Unlink()

`unlink("name");` // 在 `name` 的父目录文件中删除目录项。如果引用的 `DiskInode`, `d_link` 变成 0, 删除磁盘文件 (回收数据块和 `DiskInode`)。把它们送进 `SuperBlock` 的空闲盘块号栈和空闲 `Inode` 栈。

删除 `ast / mbox` 文件

- 线性目录搜索 `mbox` `NameI( )`, 分支 3
- 父目录 `ast` 有写权限嘛?
 - 没有, 退出
- 在 `ast` 目录文件中找到 `mbox` 对应的目录项 `myDirEntry`
 - `n = myDirEntry.m_ino`
 - `myDirEntry.m_ino = 0`
 - 修改 `n# inode`
 - `d_link - 1`, 置脏标记
 - `d_link` 是 0 嘛?
 - 是: 删除文件: 从 0# 逻辑块开始, 释放所有数据块 & 释放 `n# inode`。

ast 目录 406# 扇区

15	.
10	..
64	Grants
92	Books
9	mbox
80	temp

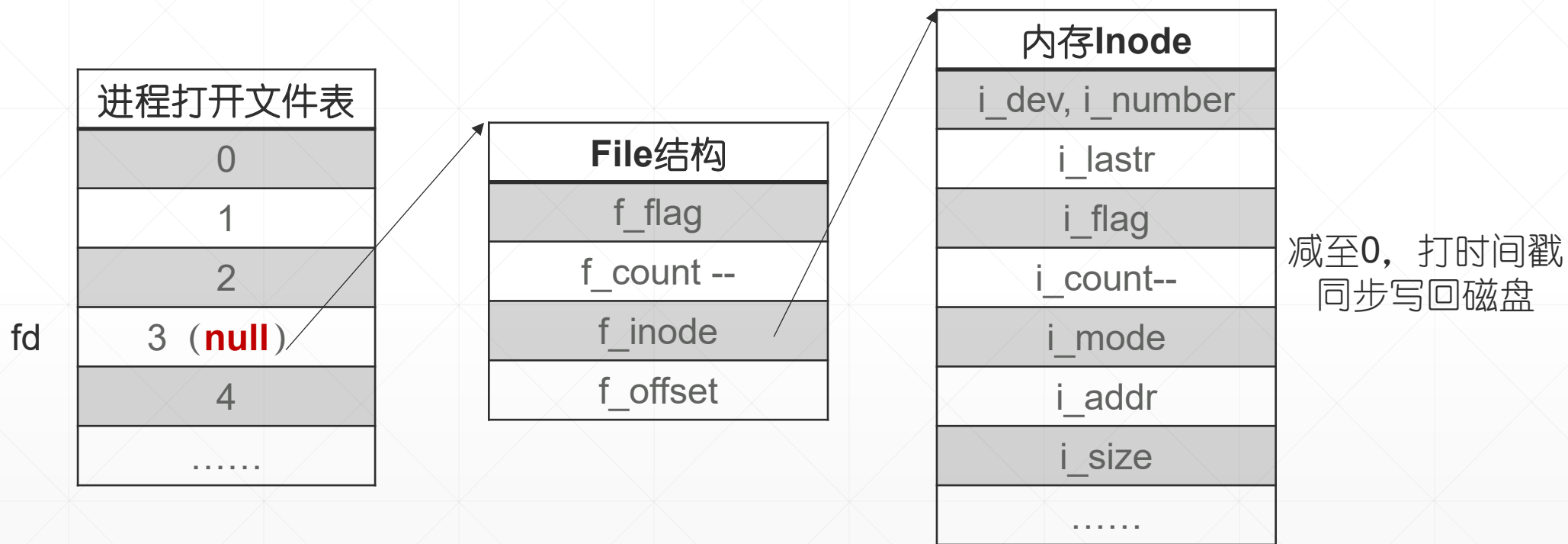
(a)

ast 目录 406# 扇区

15	.
10	..
64	Grants
92	Books
0	mbox
80	temp

(b)

七、Close(fd), 拆除打开文件结构, 释放资源





零碎的知识点 1：进程的标准输入、输出文件

0, STDIN。进程的标准输入

1, STDOUT。进程的标准输出

2, STDERR。进程的标准错误输出（V6++没有）

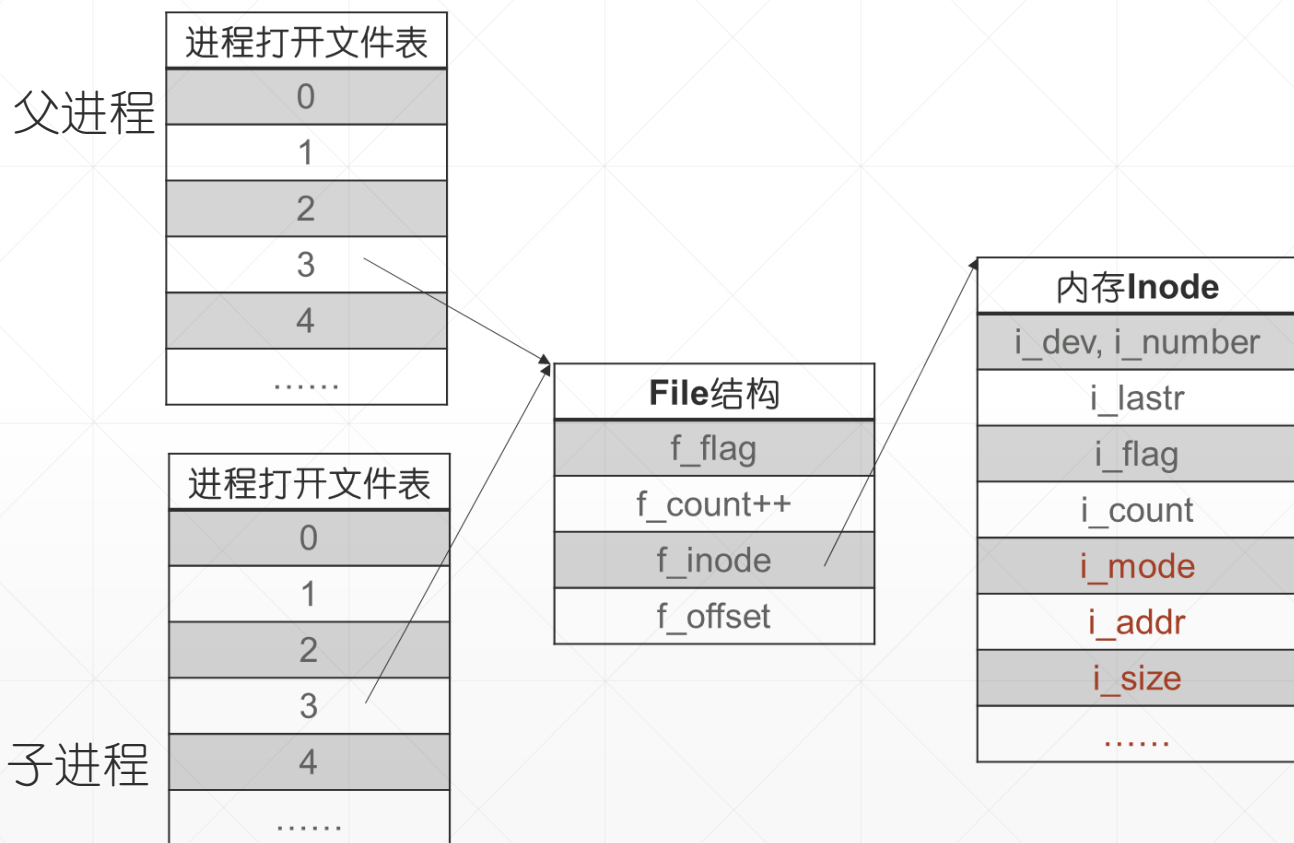
- 这三个文件无需打开。它们，用户登录（login）时打开，fork时继承自shell进程（V6++没有登录过程，系统初始化时main0打开这3个文件）



标准输入输出文件的使用

```
int main( int argc, char* argv[] )  
{  
    char c;  
    while( read(0,&c,1) == 1 )  
        write(1,&c,1);  
}
```

fork创建的子进程继承父进程打开的文件



父子进程共用一根读写指针访问文件，如果不保护，文件内容会混乱。为了防止文件内容混乱：

(1) **shell**进程创建子进程后**wait**入睡，标准输出文件供子进程（应用程序）使用。

(2) 进程创建后，父子进程应该及时关闭不使用的文件描述符。比如，管道文件读写，父进程关闭读端，子进程关闭写端。



3.4 习题

例1: ./copy file1 file2

```
int main( int argc, char* argv[] )
{
    int oldFile = open(argv[1],O_RDONLY); // oldFile是3, 是进程访问文件file1的文件描述符。读文件file1是进程的3#文件系统会话。
    int newFile = open(argv[2],O_WRONLY|O_CREAT,S_IRUSR|S_IWUSR); // oldFile是4

    char c;
    while( read(oldFile,&c,1) == 1 ) // 一次一个字节, 读 file1
        write(newFile,&c,1); // 一次一个字节, 写 file2

    close(oldFile);
    close(newFile);
}
```




copy进程使用的打开文件结构

文件描述符
oldFile
newFile

进程打开文件表	
0	
1	
2	
3	
4	
.....	

File结构 (file1)	
f_flag (读)	
f_count (1)	
f_inode	
f_offset (0)	

File结构 (file2)	
f_flag (写)	
f_count (1)	
f_inode	
f_offset (0)	

内存Inode (file1)	
i_dev, i_number	
i_lastr	
i_flag	
i_count (1)	
i_mode	
i_addr	
i_size	
.....	

内存Inode (file2)	
i_dev, i_number	
i_lastr	
i_flag	
i_count (1)	
i_mode	
i_addr	
i_size	
.....	



例2：下面程序的输出是什么？解释程序的输出。

```
int main()
{
    int fd1, fd2;

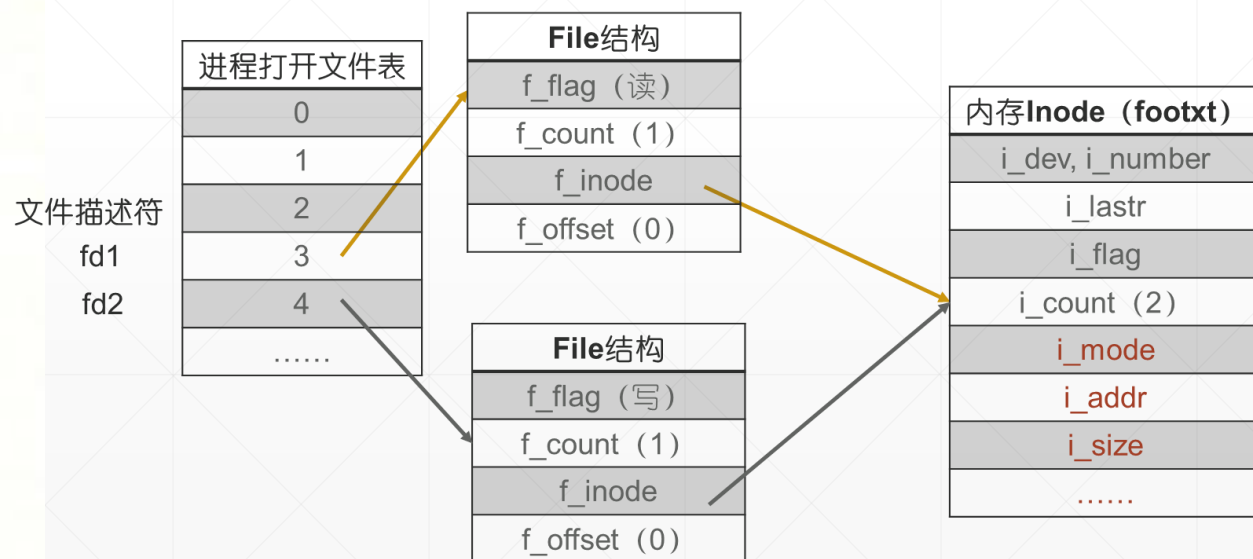
    fd1 = Open("foo.txt", O_RDONLY, 0);
    Close(fd1);
    fd2 = Open("baz.txt", O_RDONLY, 0);
    printf("fd2 = %d\n", fd2);
    exit(0);
}
```

例3：下面程序的输出是什么？解释程序的输出。

foobar文件内容： abcdefg.....

```
int main()
{
    int fd1, fd2;
    char c;

    fd1 = Open("foobar.txt", O_RDONLY, 0);
    fd2 = Open("foobar.txt", O_RDONLY, 0);
    Read(fd1, &c, 1);
    Read(fd2, &c, 1);
    printf("c = %c\n", c);
    exit(0);
}
```



例 4:

```
int main( )
```

```
{
```

```
    int fd1, fd2;
```

```
    char b;
```

```
    fd1 = open("fileA", O_RDONLY, 0);
```

```
    fd2 = open("fileA", O_RDONLY, 0);
```

```
    int count = read (fd1, &b, 1);
```

```
    printf("fd1 = %d; b = %c\n", fd1, b); // _____ (1分)
```

```
    lseek(fd1, 5, 0);
```

```
    count = read (fd1, &b, 1);
```

```
    printf("fd1 = %d; b = %c\n", fd1, b); // _____ (1分)
```

```
    count = read (fd2, &b, 1);
```

```
    printf("fd2 = %d; b = %c\n", fd2, b); // _____ (1分)
```

```
}
```

进程pa执行如下程序。假设fileA文件的内容是字符串“abcdefghijk”。

1、请在printf语句后的注释区域，下划线上写下程序的输出。

2、假设进程pa执行前，磁盘高速缓存池中没有任何关于fileA的数据。

试问：pa执行open系统调用，会入睡吗，为什么？（1分）

pa第1次执行read系统调用，会入睡吗，为什么？（1分）

pa第2次执行read系统调用，会入睡吗，为什么？（1分）

3.5 输入、输出重定向*

已知：库函数 `printf("格式化串", 输出的内容)` 的执行分 2 步 (1) 使用格式化串处理输出的内容，生成待显示的字符串 `formatted_string` (2) 执行系统调用 `←`

```
write(1, formatted_string, sizeof(formatted_string))←
```

1, 是进程的标准输出文件，默认是终端的输出缓存；即进程打开文件表 1#表项引用的 File 结构指向的是 `tty` 的内存 `inode`。 `←`

输出重定向是指修改进程的 1#文件描述符，让 `printf` 将生成的字符串写入一个指定的磁盘文件。例如：cat 命令向屏幕输出 `existFile` 文件内容。 `←`

```
$ cat existFile←
```

以下命令将 cat 程序的输出重定向至磁盘文件 `outFile`。输出重定向后，cat 命令不再向屏幕输出，原本输出的内容写入 `outFile` 文件。 `←`

```
$ cat existFile > outFile←
```



输出重定向怎么实现呢？shell 程序可以这样做：shell 进程创建一个子进程，子进程 exec(cat)之前，连续执行系统调用 close 和 open：↵

```
if ( fork( )==0 )↵
```

```
{↵
```

```
    close(1);↵
```

```
    fd = open(outFile, 写);    // fd 是 1↵
```

```
    exec(cat, ****); // cat 程序执行时标准输出（1#文件描述符）是 outFile 文件↵
```

```
}↵
```

。。。 shell 程序中父进程的分支。。。↵

习题：1、Linux系统调用dup，用来复制 File 结构。下面语句执行完毕后，newfd和oldfd引用同一个File结构。

```
newfd = dup(int oldfd);
```

(1) 请使用这个系统调用实现输出重定向。

(2) Unix V6++代码，41#系统调用dup有瑕疵，无法实现此功能，试着改进它。

2、文件系统例题和习题3，第三部分，第5题。